

RoadSense AI: System Architecture & Engineering Implementation Guide

Complete Technical Reference, Stack Specifications & Developer Manual
Software Engineering & Architecture Division

1. Technology Stack Definitions & Architectural Rationale

Technology	Category / Role	Definition & Architectural Rationale
React (Vite)	Frontend UI Framework	Component-driven single-page application framework bundled with Vite for ultra-fast HMR and development.
Node.js & Express	Backend REST Gateway	Asynchronous event-driven I/O platform handling REST endpoints, JWT auth, Multer file storage, and CORS.
MongoDB & Mongoose	Database Layer	NoSQL document database utilizing GeoJSON 2dsphere spatial indexing for rapid coordinate lookups.
Python Flask	AI Microservice	Lightweight Python REST microservice hosting PyTorch / YOLOv8 object detection runtime and inference logic.
YOLOv8 (Ultralytics)	Computer Vision Model	Deep CNN object detector trained on road defect datasets for multi-class bounding box regression and classification.
OpenCV (cv2)	Image Processing	Open-source computer vision library used to render labeled bounding boxes and confidence scores on images.
Leaflet & OpenStreetMap	GIS Mapping Layer	Open-source interactive mapping framework rendering custom severity markers, popups, and map controls.
Recharts	Analytics Visualization	SVG chart library rendering real-time municipal dashboard metrics (monthly reports, status distribution).
JWT & bcryptjs	Security & Auth	JSON Web Token stateful/stateless auth with salted bcrypt password hashing and role-based access control.

2. Predefined Municipal Officer Security ID Authentication System

To prevent unauthorized users from registering as Municipal Authorities, RoadSense AI implements a **Predefined Officer Security ID** verification system. Municipal Officers cannot self-select admin privileges; they must provide a valid ID code pre-seeded in the officeraccesscodes collection.

Predefined Security ID	Department Allocation	Authorization Level
MUN-OFFICER-8842	Public Works Engineering	Municipal Officer Admin Access
MUN-OFFICER-9913	Highway & Pavement Maintenance	Municipal Officer Admin Access
MUN-OFFICER-1045	Infrastructure Risk Assessment	Municipal Officer Admin Access
CITY-ENG-5501	Rapid Response Inspection Unit	Municipal Officer Admin Access
PWD-ADMIN-2026	Chief Municipal Operations	Chief Municipal Admin Access

3. End-to-End System Workflow & Data Flow

- Citizen Submission:** Citizen uploads road damage image and sets GPS coordinates via browser geolocation or Leaflet map picker.
- AI Processing:** Express API receives image upload via Multer and forwards absolute image path to Python Flask AI microservice (:5001/detect).

3. **Bounding Box Generation:** OpenCV draws labeled bounding boxes onto the image and returns metadata (damageType, confidence, boundingBoxes).
4. **Priority & Duplicate Calculation:** Express executes priorityCalculator.js and duplicateDetector.js using Haversine formulas.
5. **Database Persistence:** Mongoose saves the report document and triggers real-time citizen notification.
6. **Municipal Dispatch & Repair:** Municipal Officers log in using Predefined Security IDs, inspect analytics, verify reports, assign repair squads, upload completed repair photos, and mark work orders completed.

4. Step-by-Step Execution Guide

Start Production Gateway & AI Engine:

```
# Terminal 1: Backend Server (Port 5000)
cd backend && npm start

# Terminal 2: Python Flask AI Service (Port 5001)
cd ai-service && python app.py

# Access Live Application
http://localhost:5000
```